

Measuring Validator and Sandboxed-Execution Coverage for LLM-Generated NetOps Artifacts: A Paired Confirmatory Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory experiment (study_id f2c3d8b1-7a6e-4a9a-b2c3-5f8e1d2b6c7e) that measured the marginal detection benefit of adding sandboxed emulation to a deterministic validator for LLM-generated intent→configuration artifacts. Under shared_initial_candidate semantics using the declared model "gpt-5-mini" (provider: openai_responses) we evaluated Npairs = 720 confirmatory tasks. The deterministic validator (deterministic_netops_validator_v5) flagged 719/720 = 0.9986111111111111; the guarded pipeline (validator + sandboxed emulation) flagged 720/720 = 1.0. Paired difference (guarded - baseline) = 0.001388888888888884; exact McNemar two-sided p = 1.0; 95% bootstrap percentile CI (10000 resamples, seed = 1729) = [0.0, 0.004166666666666667]. Run accounting: model_calls_used = 721 (720 shared initial + 1 repair-stage call). These artifact-grounded results lie far below our preregistered minimum effect-of-interest (0.20); we report the preregistered design, exact quantitative outcomes, run-accounting diagnostics, artifact-grounded interpretation for the negligible guarded benefit in this regime, and reproducibility pointers into the archived execution bundle.

I. INTRODUCTION

Generative large language models (LLMs) are increasingly proposed to translate operator intent into device configuration and NetOps workflows. Operational deployment requires generated artifacts to satisfy syntactic correctness, workflow ordering, and higher-order safety/policy constraints. A common mitigation architecture is generate→validate→(repair)→execute, sometimes augmented by sandboxed emulation to reveal runtime-only failures that static validators miss. Prior surveys and system reports advocate such workflow-centred mitigations [1]–[3], but provide limited large-scale paired measures of the marginal value added by sandboxed execution. Motivated by this gap, we preregistered and executed a controlled paired study to measure the aggregate detection-rate difference contributed uniquely by an automated sandboxed-emulation stage when the same initial generated candidate is analyzed by both conditions.

Research question and contributions. Our preregistered research question was: for synthetic intent→configuration tasks under shared_initial_candidate semantics, what is the paired detection-rate difference (guarded - baseline) between (A) a deterministic validator-only pipeline and (B) the same deterministic validator followed by automated sandboxed emulation? Contributions: (1) a machine-readable preregistration and faithful autonomous execution; (2) an artifact-grounded

paired analysis on Npairs = 720 with complete accounting; (3) an artifact-grounded interpretation explaining a near-zero marginal effect; (4) concise reproducibility pointers into the archived bundle and prescriptive boundary conditions where guarded stages are likely to matter.

II. RELATED WORK

Workflow-centred mitigations and verification tools. Recent surveys and architecting reports document a common pattern for LLM-enabled NetOps: pair generation with deterministic validators, formal verifiers, or emulation to reduce unsafe or incorrect device-level actions [1], [2]. Tool- and system-focused work demonstrates concrete instantiations: configuration-analysis engines (e.g., Batfish-style approaches and NetKAT-based verifiers) provide rule-driven semantic checks over device configurations and have been extended in prototypes that combine generation with symbolic or executable verification [2], [3]. Parallel lines of work emphasize the role of sandboxed execution (Mininet/Mininet+GNS3 hybrids, lightweight emulators) to surface dynamic, stateful failures that static validators cannot express, particularly for make-before-break or multi-step state transitions [4], [5].

Coverage heterogeneity and verifier ceilings. Prior empirical and methodological studies highlight that validator gains depend strongly on the validator rule-set, the task abstraction, and the emulator fidelity. For instance, verification-oriented projects show that many common configuration errors (syntactic mistakes, missing required commands, simple ordering violations) are well-covered by high-quality static validators, whereas subtle runtime invariants or environment-dependent failures (timing-dependent race conditions, stateful interaction errors) often escape static analyses and require emulation or more expressive specification languages [2], [6]. This heterogeneity creates two operationally important regimes: (i) validator-dominated regimes where a broad, rule-rich validator attains a high true-positive rate and leaves little headroom for emulation to add detections, and (ii) runtime-detection regimes where emulators or high-fidelity execution are necessary to reveal violations that are intrinsically dynamic.

LLM interaction patterns, mistake localization, and repair. Empirical studies in adjacent program- and specification-generation tasks show a recurring pattern: LLMs may fail to locate their own reasoning errors reliably but can perform targeted corrections when errors or locations are externally

signalled or when a verifier pinpoints a violation [5], [7]. This motivates generate→validate→repair pipelines that use validators or emulators as external mistake detectors and—optionally—invoke constrained repair-stage model calls. The present study enforces shared_initial_candidate semantics to isolate exactly these downstream contributions (validator vs. emulator vs. explicitly recorded repair calls) and avoid conflating generation variability with guarded-stage effects.

Benchmarks, evaluation gaps, and operational implications. Several surveys and position papers argue that existing LLM benchmarks insufficiently capture workflow-centred safety properties (rollback-aware scoring, multi-device transactional integrity, emulation-confirmed invariants) and call for niche benchmarks that combine intent→generate, static verification, and sandboxed execution to reflect operational trustworthiness [1], [6]. Our confirmatory paired design directly addresses this suggested evaluation axis by quantifying the marginal detection contribution of an emulation stage under tightly controlled shared-candidate semantics; the negligible aggregate gain we observed is consistent with a validator-ceiling interpretation in a curated synthetic task bank and illustrates the evaluation-risk of reporting single-stage improvements without workflow-centred controls.

III. METHODOLOGY

Preregistration and analytic intent. We preregistered a confirmatory paired-binary design (study_id f2c3d8b1-7a6e-4a9a-b2c3-5f8e1d2b6c7e). The primary estimand was the paired_success_rate_difference_guarded_minus_baseline aggregated across confirmatory samples; the preregistered minimum effect-of-interest for power planning was 0.20 absolute. The preregistration (frozen prior to execution) specified: inclusion/exclusion rules, deterministic missingness and reseed policies (up to two automatic reseeds for parser failures), repair-stage limits (≤ 1 repair call per task), contamination detection and deterministic replacement rules, and the analysis executor (paired_binary_analysis_v1) with paired inference procedures and pre-registered sensitivity analyses.

Task bank, topology templates, and vendor abstraction. Confirmatory items were synthetic intent→configuration tasks generated from the intent_configuration_repair_v1 family. The bank covered three abstracted vendor-CLI families and three topology templates (leaf–spine, 3-node service chain, simple triangle). Each task manifest encoded: the initial device state, an explicit intent, a required command sequence, allowed-touched fields, and parser-acceptance constraints. Inclusion required vendor-specific parser acceptance after up to two deterministic reseeds; irrecoverable parser failures were to be deterministically excluded and replaced per the preregistered missingness plan. In the executed run there were zero irrecoverable parser exclusions and complete_pair_count = 720 (analysis/missingness_summary.csv).

Model interface, sampling semantics, and shared_initial_candidate enforcement. The declared/requested model was recorded as "gpt-5-mini" and the execution model_configuration.json contains declared_model_version =

"gpt-5-mini", max_output_tokens = 2000, and temperature = null (sha256 a40e96e212ab87da...). Important operational clarification (preregistered and enforced): temperature = null in the archived configuration indicates the client did not set an explicit temperature parameter; null does not imply temperature=0 or provider-side determinism. To isolate guarded-stage effects we enforced shared_initial_candidate semantics: for each pair the pipeline performed a single sampled initial generation (shared) which both the baseline validator and the guarded pipeline consumed. Concretely, provider_call_audit and deterministic_reconciliation artifacts record stage_counts: shared_initial_generation = 720 and repair = 1, and model_calls_used = 721 (720 shared + one repair-stage call). Because the same sampled candidate was analyzed by both conditions for each pair, any between-condition discordance must stem from (i) guarded-stage sandboxed emulation observations not encoded as validator violations, and/or (ii) the permitted repair-stage intervention recorded in run accounting.

Validator implementation and scoring rule. Baseline validation used deterministic_netops_validator_v5 (scorer_version = 5.0) executing the preregistered scoring rule full_intent_constraint_validation. The validator performs multi-level checks recorded in per-episode validator_trace objects: vendor-specific parsing, syntactic command parsing, command ordering/workflow checks against required_sequence, required-field presence, dependency constraints (e.g., must-shutdown-before-mtu-change patterns), and policy constraints (protected interfaces, group-wise minimum-active constraints). The validator emits structured outputs used in scoring: normalized_configuration, parsed_command_count, required_command_count, observed_touched_field_count, violation_count, and violation_codes. Per-episode artifacts (execution/scoring/task-XXXXX-*.json) show these fields and validator_version = 5.0 for all confirmatory episodes.

Sandboxed-emulation harness and guarded pipeline semantics. The guarded pipeline applied the baseline deterministic validator and then, for candidates not blocked by parser failure, executed an automated sandboxed-emulation harness that instantiated an emulated representation of the declared topology and applied the candidate commands. The emulation harness produced runtime observations that were normalized and summarized into the same validator-trace structure (validation_before/validation_after, final_state, parsed_command_count, observed_touched_field_count). The emulation stage was designed to detect runtime-only symptoms such as transient ordering-dependent state changes, reachability/regression signals, or state transitions that static rules do not explicitly capture. The preregistered repair policy permitted up to one repair-stage model call per task; repair calls (if used) and their provider traces are recorded in the execution manifest and deterministic_reconciliation artifacts.

Inclusion/exclusion, missingness, and contamination controls. The preregistered missingness plan allowed two automatic deterministic reseeds for parser failures; irrecover-

able parser failures would be deterministically replaced. Contamination controls included deterministic SHA-256 artifact hashing, automated prompt-template divergence checks, and validator/emulator binary/config checksum verification before confirmatory execution. The execution manifest and analysis/contamination_summary.csv report no contamination flags and no irrecoverable parser exclusions for the confirmatory set.

Statistical analysis procedures. The primary confirmatory analysis used the paired_binary_analysis_v1 executor specified in the preregistration. The primary test computed the aggregate paired detection-rate difference (guarded - baseline) across complete pairs and reported an exact McNemar two-sided p-value. Pre-registered uncertainty quantification used paired-bootstrap percentile confidence intervals with 10,000 resamples and a fixed seed (bootstrap_seed = 1729) for reproducibility. Secondary exploratory analyses (per-class and per-vendor paired differences) were pre-registered with Bonferroni adjustments; these are archived but not the focus of the confirmatory report. The preregistered default treatment of irrecoverable technical failures for the primary analysis was to exclude complete-pair technical losses (complete_pair_primary); a sensitivity analysis treats irrecoverable failures as nondetections (documented in the preregistration). The executed analysis followed the preregistered complete_pair_primary plan; analysis artifacts and logs are archived (analysis/analysis_log.jsonl and analysis/paired_contingency_table.csv).

Deterministic reconciliation and run accounting. To enable artifact-grounded claims and independent checking we recorded provider_call_audit and deterministic_reconciliation artifacts. These reconcile: planned_episode_count = 1440, completed_episode_count = 1440, complete_pair_count = 720, failed_episode_count = 0, model_calls_used = 721, and stage_counts (shared_initial_generation = 720; repair = 1). These machine-verifiable run-accounting artifacts are referenced throughout the manuscript when we state numerical outcomes and when tracing the single discordant guarded-only detection back to a documented repair-stage invocation.

IV. RESULTS

Primary confirmatory counts and test statistics (artifact-grounded). Analysis artifacts report the paired contingency counts: n11 = 719 (detected by both), n10 = 1 (guarded-only), n01 = 0 (baseline-only), n00 = 0 (neither). Marginal detection rates are baseline = $719/720 = 0.9986111111111111$ and guarded = $720/720 = 1.0$. The paired estimate (guarded - baseline) = 0.0013888888888888884. Exact McNemar two-sided test p = 1.0. The paired-bootstrap percentile 95% CI (10000 resamples, bootstrap_seed = 1729) = [0.0, 0.004166666666666667]. These numbers are reproduced verbatim from analysis/condition_summary.csv (sha256 394bc690671b731194f9b42b4dcbe28fded6e2ec8cf898b79831225343c22c28) and analysis/paired_contingency_table.csv (sha256 efe6e8e7016fa843a8226e911dbde829e943903d719101da8d956b3ff899133c).

Canonical artifact pointers and the unique discordant episode. The complete per-episode raw results are archived in execution/raw_results.jsonl (input-results SHA-256 = df367ecc40a6f0eaf402129260e11d8450884a4ea5fdb94a602e0d2513aef0e8). The single discordant pair (the unique n10 item) is the lone entry in execution/raw_results.jsonl whose recorded baseline_score == 0 and guarded_score == 1. That raw_results entry contains the canonical pair_id field and includes the validator_trace and repair_provider_trace fields that link the guarded detection to a downstream repair-stage provider trace. Deterministic reconciliation and provider-call accounting are archived in analysis/deterministic_reconciliation.json (sha256 49731a2e52e049ccf5b53c35ddac2351ad75e7e95504dc4a42bcb096d0dec3aa) and confirm model_calls_used = 721 and stage_counts = {"shared_initial_generation": 720, "repair": 1}. In short: the discordant episode is uniquely identifiable within the archived bundle (execution/raw_results.jsonl, sha256 df367ecc...) and the same bundle contains the deterministic_reconciliation artifact (sha256 49731a2e5...) that documents the single repair-stage call that is connected to that raw_results entry via its repair_provider_trace field.

Run-accounting diagnostics and sensitivity checks. Provider-call audit (deterministic_reconciliation.json) reports hosted_model_call_event_count = 721, completed_hosted_model_call_event_count = 721, failed_hosted_model_call_event_count = 0, cache_hit_count = 0, cache_miss_count = 721. The preregistered missingness policy and sensitivity analyses were applied; no irrecoverable parser or emulation failures occurred (analysis/missingness_summary.csv documents complete_pairs = 720 and zeros for all failure categories). The primary analysis used the preregistered complete_pair_primary treatment; the preplanned sensitivity (treating irrecoverable technical failures as nondetections) is moot here because there were no such failures in the confirmatory run.

Representative per-episode diagnostics. Representative scoring traces (execution/scoring/task-000001-*.json ... task-000006-*.json) show parsed_command_count values of 4, 2, 6, 4, 3, and 6 respectively and validator violation_count = 0 for these examples; the validator_version reported in these traces is "5.0". These representative artifacts illustrate that the shared_initial candidates in the confirmatory bank were typically complete with respect to the required command sequences and that validator checks reported validity for the majority of cases. The representative traces are archived in the execution bundle and are consistent with the high baseline success rate.

Attribution of the single discordant detection. Because shared_initial_candidate semantics were enforced (one shared sampled initial candidate per pair), the only proximate causes for the single n10 observation are (A) an emulation observation not encoded as a validator rule that produced a guarded-stage detection, or (B) the single recorded repair-stage invocation documented in provider-call accounting. The raw_results entry for the discordant pair includes the repair_provider_trace

and `validator_trace` fields permitting an auditor to follow the exact artifact-level linkage from the guarded detection to the recorded repair-stage call; both the `raw_results` file (`sha256 df367ecc...`) and the reconciliation artifact (`sha256 49731a2e5...`) are present in the canonical execution bundle.

Implications of the quantitative diagnostics. The observed discordant count ($n_{10} = 1$) yields an aggregate paired difference that is two orders of magnitude below our preregistered minimum effect-of-interest (0.20). The bootstrap CI [0.0, 0.004166666666666667] quantifies the narrowness of the realized uncertainty in this curated regime. Combined with run-accounting (`model_calls_used` = 721; `repair` = 1) and the representative per-episode diagnostics, the archived evidence supports a ceiling-effect interpretation: the deterministic validator covered the task bank’s explicit syntactic and workflow constraints so thoroughly that the guarded-stage emulation had almost no additional aggregate detections to surface.

V. DISCUSSION

Why was the guarded-stage aggregate benefit negligible? The archived evidence supports three artifact-grounded factors.

1) Validator ceiling effect. Deterministic `netops_validator_v5` covered the syntactic and explicit semantic properties encoded in the synthetic task bank (shutdown—configure—restore, make-before-break uplink switchover, paired MTU changes). Baseline success rate ≈ 0.9986 indicates near-universal validator coverage for these curated patterns, leaving negligible headroom for the guarded stage.

2) Task curation and inclusion policy. Confirmatory items were parser-validated synthetic tasks specifically designed to exercise workflow policies but not constructed adversarially to evade validator checks. This curation concentrates cases inside the validator’s coverage envelope and reduces the prevalence of subtle runtime-only failure modes that emulation might reveal.

3) Shared `initial_candidate` semantics and limited repair activity. Enforcing one sampled initial candidate per pair isolates guarded-stage contributions but removes between-condition generation variability. `Provider_call_audit` and `deterministic_reconciliation.json` record one repair-stage invocation aligned with the single n_{10} discordant count; thus the observed discordance is attributable to downstream guarded-stage activity recorded in run accounting rather than generation differences.

Operational implications and boundary conditions. For environments where a high-coverage static validator exists and inputs match its expected patterns (parser-validated, constrained intents), adding automated sandboxed emulation may provide negligible aggregate additional detection at scale. This conditional conclusion must not be generalized: in operational regimes with noisier natural-language intents, adversarial inputs, broader vendor diversity, stateful multi-step interactions, or validator gaps, guarded runtime checks and higher-fidelity emulation are likely to provide substantive benefit. The archived per-episode traces support targeted follow-

up hypotheses: adversarial task injection, broader vendor-CLI coverage, and complementary validator families should increase the guarded-stage marginal contribution.

VI. LIMITATIONS

- Synthetic task bank and deterministic task generator produce limited ecological diversity and create a validator-ceiling effect; results may not generalize to noisier, adversarial, or production distributions.
- Single declared/requested model family (`gpt-5-mini`) and single validator implementation (`deterministic_netops_validator_v5`) limit generalizability across models and validator families.
- Preregistered shared `initial_candidate` semantics (one initial generation reused across paired conditions) isolate guarded-stage contribution but remove between-condition generation variability; this design choice constrains discordance sources to downstream guarded-stage observations or repair calls.
- Sandboxed-emulation fidelity relative to vendor/OS production behaviors and large-scale stateful interactions is limited by the emulation harness used; higher-fidelity emulators could change outcomes in other regimes.
- Study power planning targeted a minimum detectable paired difference of 0.20; the observed aggregate effect (~ 0.0014) lies far below that design sensitivity and should be interpreted as a narrowly scoped near-zero finding for this experimental regime rather than a general negative result for all NetOps workflows.
- Provider-side nondeterminism and hidden caching remain residual uncertainties despite deterministic reconciliation procedures; these are documented in `analysis/deterministic_reconciliation.json` and `provider_call_audit` artifacts.

VII. CONCLUSION

We preregistered and executed a paired confirmatory study comparing a strong deterministic validator with the same validator augmented by sandboxed emulation on a curated synthetic intent→configuration task bank. Over 720 paired tasks the guarded pipeline produced a negligible aggregate detection gain (0.0013888888888888884) with exact McNemar $p = 1.0$ and 95% bootstrap CI [0.0, 0.004166666666666667]. Run accounting shows one repair-stage invocation corresponding to the single guarded-only detection; because shared `initial_candidate` semantics were enforced, archived per-episode traces permit independent verification of that mapping via `execution/raw_results.jsonl` and `analysis/deterministic_reconciliation.json`. We interpret the result as arising from validator ceiling effects and curated task design; follow-up work should focus on adversarial and production-like task banks, multi-validator ensembles, and higher-fidelity emulation to identify regimes where guarded stages add substantive operational value.

Reproducibility pointers (compact). The canonical execution bundle archived by the run contains:

execution/execution_manifest.json (execution summary and preregistration SHA-256), execution/raw_results.jsonl (input-results SHA-256 df367ecc40a6f0eaf402129260e11d8450884a4ea5fdb94a602e0d2513aef0e8), analysis/deterministic_reconciliation.json (sha256 49731a2e52e049ccf5b53c35ddac2351ad75e7e95504dc4a42bcb096d0dec3aa), execution/model_configuration.json (sha256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820), analysis/paired_contingency_table.csv (sha256 efe6e8e7016fa843a8226e911dbde829e943903d719101da8d956b3ff899133c), and analysis/condition_summary.csv (sha256 394bc690671b731194f9b42b4dcbe28fdd6e2ec8cf898b79831225343c22c28).

The discordant episode is the unique raw_results.jsonl entry with baseline_score = 0 and guarded_score = 1; that record contains the pair_id and validator/emulation traces that map it to the single repair-stage invocation recorded in deterministic_reconciliation.json. These artifacts are archived as the canonical reproducibility bundle referenced by the execution manifest and the preregistration record. (See Disclosure for exact manifest references.)

DISCLOSURE STATEMENT

Disclosure: Autonomous post-lock research run executed under the frozen master prompt supplied to the system. Declared/requested model: "gpt-5-mini" (provider recorded as openai_responses). Deterministic validator: deterministic_netops_validator_v5. Orchestration adapter family: hosted_netops_gvr_v1. Preregistration id: f2c3d8b1-7a6e-4a9a-b2c3-5f8e1d2b6c7e (preregistration SHA-256: 0169919fb8c5aedb2c5a78e031f1ab5c3aeda405cde1ae80fb2f864768595c1b; recorded in the execution manifest). Initial master-prompt immutable reference: provenance/master_prompt.txt, SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba. Execution summary (artifact-grounded): completed_episode_count = 1440, complete_pair_count = 720, model_calls_used = 721 (720 shared initial + 1 repair-stage call); stage_counts and provider-call audit are archived in analysis/deterministic_reconciliation.json (sha256 49731a2e52e0...). Human role and autonomy boundary: a human Research Director selected the broad NetOps topic family and constrained the pre-lock master prompt; after the autonomy lock the agentic pipeline autonomously performed literature selection, preregistration, code and prompt generation, experiment execution, analysis, peer review, and manuscript drafting. No manual human editing of technical manuscript content occurred after the autonomy lock. The execution manifest and archived artifacts named above serve as the canonical reproducibility bundle.

REFERENCES

- [1] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026, 10.48550/arxiv.2605.12729
- [2] Matt Brown, Ari Fogel, Daniel Halperin, Victor Heorhiadi, Ratul Mahajan, Todd Millstein, "Lessons from the evolution of the Batfish configuration analysis tool", 2023, 10.1145/3603269.3604866

[3] <http://arxiv.org/abs/2407.08249>